Spring Boot

Développer des applications d’entreprise

Table des matières

- Le conteneur Spring
  - Pratiques de conception
  - Spring dans l'écosystème Java EE
  - Les composants essentiels
    - Core
    - DAL
    - MVC
    - WebFlux
  - Intégration à d’autre technologie
  - Les stratégies de déploiement

Tables des matières

- Gestion des beans
  - Découpage en couches, l’approche POJO
  - La gestion de l’état et l’injection de dépendances,
  - Les intercepteurs et le programmation orientée aspect
  - Le cache et la supervision avec JMX
  - Les tâches et scheduling
  - Les profils
  - Les test des beans Spring

Table des matières

- Accès aux données et gestion des transactions
  - Couplages avec différentes technologies
    - JDBC
    - JPA
    - NoSQL
  - La gestion des transactions
  - Le support des transactions au sein des tests

Table des matières

- API REST avec Spring MVC et Spring WebFlux
  - Bonnes pratiques
  - Les bases du protocole HTTP
  - Mise en place de l’API REST
  - L’API Bean validation
  - La gestion des exceptions
  - La programmation réactive avec Spring Webflux

Table des matières

- Spring Security
  - Choix d’un référentiel utilisateur
  - Mode d’authentification
    - Session
    - Token
  - Sécurité applicative
  - Sécurisation des routes

Table des matières

- Échange de messages avec Spring WebSocket
  - Design Pattern
  - Le principe de publish/subscribe
  - Presentation de STOMP et SockJS
  - Support et mécanisme de fallback

Table des matières

- IHM Web avec Spring MVC
  - Validation
  - Rappel du pattern MVC
  - Internationalisation
  - Gestion des exceptions

Table des matières

- Packaging et déploiement avec Spring Boot
  - La gestion des dépendances
  - Les starters
  - La configuration automatique
  - L’environnement d’exécution
  - Le packaging
    - Jar
    - War
    - Image OCI
  - Le déploiement

Pratique de conception

Le conteneur Spring - Pratique de conception

Dans l’écosystème Java EE

Le conteneur Spring - Spring dans l'écosystème Java EE

Spring est un framework apparu en 2004 en tant que conteneur d’objet. Il a depuis évolué.

- Spring MVC
- Spring Data
- Spring Batch
- …

Le conteneur Spring - Spring dans l'écosystème Java EE

Quelques points de repère:

- Context => Structure contenant les singletons permettant le partage d’objet
  - Spring permet l’accès à ces objets grâce à leur nom, leur type ou bien les deux
  - Spring effectue la création et la gestion des dépendances éventuelles

- Application en couche et programmation par interface
  - Spring facilite l’utilisation d’application en couche distincte
    - DAL - Business - Presentation

Le conteneur Spring - Spring dans l'écosystème Java EE

- La déclaration de ces singletons peut être faite de trois façon, qui peuvent éventuellement être mixées:
  - Fichier XML
  - Classe de configuration
  - Annotation

Le conteneur Spring - Spring dans l’écosystème Java EE

Configuration par XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
        http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- bean definitions here -->
    <bean id="customerCreate" class="be.bstorm.formation.slidejpa.models.dtos.customer.CustomerCreate" />
</beans>
```

Le conteneur Spring - Spring dans l’écosystème Java EE

Configuration par Classe

```
@Configuration
public class BeanConfiguration {

    @Bean
    public CustomerCreate customerCreate() {
        return new CustomerCreate();
    }
}
```

Le conteneur Spring - Spring dans l’écosystème Java EE

Configuration par Annotation

```
3 usages
@Component
public class Appartement {

    // champs
    @Autowired
    @Qualifier("person_02")
    private Personne proprietaire;
```

```
6   @Component
7   @Qualifier(value = "person_02")
8   public class Personne {
9
10
11
12
```

Les composants essentiels

Les composants essentiels - Core

Spring Core est un package constitutif du framework Spring.
Celui-ci contient:

- Le module de Dependency Injection
- Le module de gestion d'événement
- Le module de gestion des ressources
- Le module d’internationalisation
- Le module de validation
- Le module de Data Binding
- Le module de conversion de type
- Le module de Spring Expression Language (SpEL)
- Le module de Programmation Orienté-Aspect (AOP)

Les composants essentiels - Accès aux données

Le composant Spring Data Access est un package constitutif de Spring, il contient:

- Un module permettant la gestion des transactions
- Un module permettant la gestion des DAO
- Un module permettant l’utilisation d’un ORM
- Un module de configuration par XML et annotation

Les composants essentiels - MVC

Le composant MVC est la version originale du framework construit sur base de l’API Servlet.

Comme tout framework Web développé en Java, il est construit autour d’un front controller qui réceptionne tous les appels au serveur avant des les dispatchers.

Sur Spring Web MVC, ce contrôleur est appelé DispatcherServlet

Les composants essentiels - WebFlux

Le composant Spring WebFlux fut introduit dans la version 5.0 du framework.

Ce composant permet l'utilisation d'un mode non bloquant.

Il fournit un support complet de Reactive Streams (https://www.reactive-streams.org/). La librairie utilisé par Spring est Reactor (https://projectreactor.io/)

Les composants essentiels - Intégration

RestEndpoints

- RestTemplate
- WebClient

Services Web

- AMQP
- Java Web Services

EJB

Java Message Service (JMS)

Java Management Extensions

EMAIL

Execution de tâche est scheduling

Gestion de cache

Les stratégies de déploiement

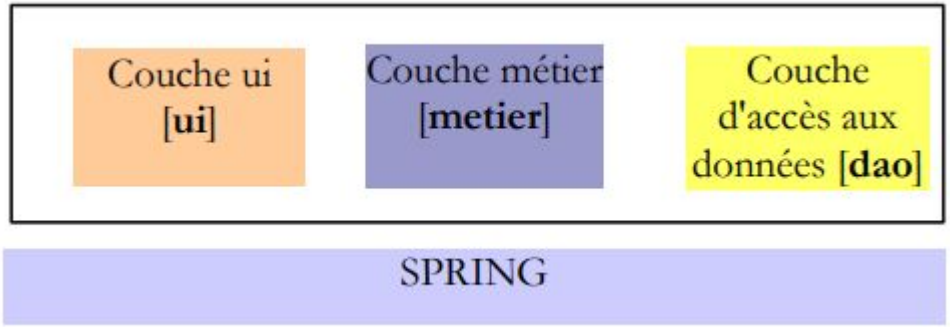
Le conteneur Spring - Les stratégies de déploiement

- Jar
  - Par défaut, Spring utilise la méthode “public static void main” comme point d’entrée qui lancera le server web pour vous.
  - Le serveur par défaut est un Tomcat 7
  - Il est possible d’utiliser Jetty
    - Exclure la dépendance par défaut de Tomcat
    - Include la dépendance Jetty
- War
  - Permet l’installation d’une application Spring sur un server Tomcat déjà existant
    - Modification de la configuration de Maven/Gradle
- Dans le cloud
  - Heroku
  - Cloud Foundry
- Docker container

Gestion des beans avec Spring Core

Le découpage en couche, l’approche POJO

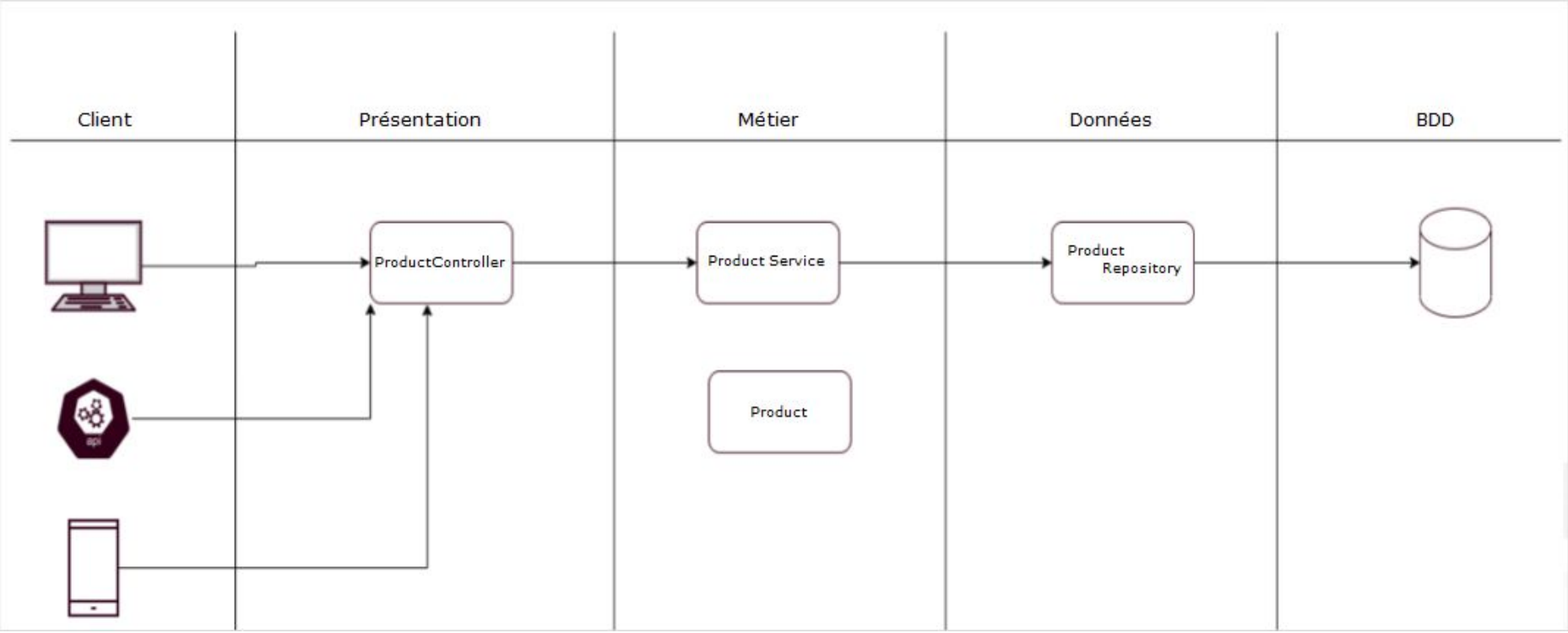
Gestion des beans - Le découpage en couche



Gestion des beans - Le découpage en couche

- La couche de Présentation:
  - Il s’agit de l’interface accessible par l’utilisateur
  - Il peut s’agir:
    - Page web
    - Interface console
    - …
- La couche Métier:
  - Il s’agit de l’interface permettant l’implémentation de la logique métier de l’application
  - Il peut y avoir plusieurs implémentation possible pour un même métier
- La couche DAL:
  - Il s’agit de la couche d’accès au données.

Gestion des beans - Le découpage en couche



Gestion des Beans - Les intercepteurs

Gestion des beans - Les intercepteurs

Les intercepteurs sont des classes Java permettant l'interception des requêtes http.
Vous pouvez utiliser ces intercepteurs à deux moments:

- Avant d'envoyer la requête au contrôleur
- Avant d'envoyer la réponse au client

Leurs rôles est d'effectuer un traitement sur ces requêtes:

- Autorisation
- Log
- …

Gestion des Beans - Les intercepteurs

- La méthode “*preHandle*” permet d’effectuer du traitement avant le controller
- La méthode “*postHandle*” permet d’effectuer du traitement avant l’envoie au client
- La méthode “*afterCompletion*” permet d’effectuer du traitement lorsque tout a été traiter

```java
public class SlideInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler) throws Exception {
        return true;
    }

    @Override
    public void postHandle(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler,
            ModelAndView modelAndView) throws Exception {
    }

    @Override
    public void afterCompletion(
            HttpServletRequest request,
            HttpServletResponse response,
            Object handler,
            Exception ex) throws Exception {

    }
}
```

Gestion des Beans - Les intercepteurs

Ci-dessous le résultat d'un ***interceptor*** permettant de loguer des requêtes.

Cas d'utilisation réel:

- Gestion de session
- Envoi d'email & validation db
- Authentication & authorisation
- Calcul de temps d'exécution / performance

```
b.b.o.s.c.interceptors.SlideInterceptor  : [preHandle] => [org.apache.catalina.connector.RequestFacade@3fb6875] [GET]: /?params1=Blop
b.b.o.s.c.interceptors.SlideInterceptor  : [postHandler] => [org.apache.catalina.connector.RequestFacade@3fb6875]
b.b.o.s.c.interceptors.SlideInterceptor  : [afterCompletion] => [org.apache.catalina.connector.RequestFacade@3fb6875] [exception: null]
```

Gestion des Beans - POA

Gestion des Beans - Programmation orientée aspect

Un aspect représente une catégorie d’action à réaliser sous certaines conditions.

Ce paradigme permet, entre autres:

- Gérer les transactions
- Gérer les logs
- Gérer la mise en cache
- …

Gestion des Beans - AOP: Terminologie

- Aspect

  Problématique que l’on veut ajouter transversalement

- JoinPoint

  Le point du flux où l’aspect sera ajouté

- Advice

  L’action de l’aspect à ajouter

Gestion des Beans - AOP

- PointCut

  Expression qui définit l’ensemble des JoinPoint éligibles pour l’Advice

- Target object

  L’objet sur lequel est appliqué l’aspect

- Weaving

  Le processus qui permet de réaliser l’insertion d’aspect. Soit au moment de la compilation (AOP Statique) soit au moment de l’exécution (AOP Dynamique)

  Remarque :

  Dans le cas de Spring AOP, cela se fait au moment de la création du contexte de l’application

Gestion des Beans - AOP dans Spring

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Gestion des Beans - AOP hors Spring Boot

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-aop</artifactId>
    <version>5.3.1</version>
</dependency>

<dependency>
    <groupId>org.aspectj</groupId>
    <artifactId>aspectjrt</artifactId>
    <version>1.9.6</version>
</dependency>

<dependency>
    <groupId>org.aspectj</groupId>
    <artifactId>aspectjweaver</artifactId>
    <version>1.9.6</version>
</dependency>
```

Gestion des Beans - AOP dans Spring

```java
@Slf4j
@Aspect
@Component
public class LogAspect {

    @Before(value = "execution(public * be.bstorm.orsys.slide.services.*.*(..))")
    public void log(JoinPoint joinPoint) {
        StringBuilder builder = new StringBuilder();
        builder
                .append(joinPoint.toShortString())
                .append("(")
                .append(Arrays.stream(joinPoint.getArgs()).map(Object::toString).collect(Collectors.joining( delimiter: ", ")))
                .append(")");
        log.info(builder.toString());
    }
}
```

Gestion des Beans - AOP dans Spring

Dans l’exemple précédent, nous avons créé un Aspect.
Il est nécessaire d'annoter la classe par

- ➔ *@Aspect* : permet de définir un aspect
- ➔ *@Component* : permet la prise en charge par Spring

La méthode “log” est l’ ”Advice”. Il est annoté par “@Before” qui indique à l’ “AOP” que cet Advice doit s’appliquer avant l’appel à la méthode.

Cela correspond donc à notre “Point Cut”

Remarque :

Tout “Point Cut” doit commencer par un “designator” qui établit la cible.

Gestion des Beans - AOP: Designator

Designator	Target
execution()	Méthode décrite
within()	Méthode d’un type décrit
target()	Méthode d’un type donné
args()	Méthode dont les types de paramètres correspondent
@target()	Méthode d’un type portant l’annotation
@args()	Méthode dont les arguments portent les annotations
@within()	Méthode d’un type portant l’annotation
@annotation()	Méthode portant elle-même l’annotation

Gestion des Beans - AOP dans Spring

```java
@Around(value = "@annotation(supervision)")
public Object superviser(ProceedingJoinPoint joinPoint, Supervision supervision) throws Throwable {
    long maxTime = supervision.timeInMillis();
    long start = System.currentTimeMillis();
    try {
        return joinPoint.proceed(joinPoint.getArgs());
    } finally {
        long end = System.currentTimeMillis();
        long time = end -start;
        if (time > maxTime) {
            log.warn("[{}] spent too much time => {}", joinPoint.toShortString(), time);
        }
    }
}
```

Gestion des Beans - AOP dans Spring

Dans l’exemple ci-dessus, nous voyons comment nous pouvons répondre, grâce à des logs, à des contraintes non-fonctionnelles tel que le temps d’exécution.

L’annotation “@Around” permet à un Aspect d’englober l’appel d’une méthode

Gestion des Beans - Cache

Gestion des Beans - Cache

Mettre en cache consiste à mettre une partie des données dans une mémoire temporaire (généralement la RAM). Elle agit comme un lien entre la DAL et l’application afin de réduire le nombre d'appels à la base de données.

<u>Que met-on dans la cache ?</u>

Les données dont la modification n’est pas fréquente ou les données fréquemment lue sans modification sur une période données

Gestion des Beans - Cache

Il existe plusieurs type de cache:

- In-memory
  - Augmente les performances de l’application => Memcached ou Redis en sont de bon exemples
- Database
  - Permet le web dynamique. Utiliser dans les applications Multi-Tiers. Améliore la mise à l'échelle et les performances en distribuant la charge de travail => Hibernate
- Web Server
  - Mettre en cache la page que l’utilisateur visite afin d’éviter la surcharge en cas de nouvelle requête de la part de l’utilisateur
- CDN
  - Permet la réplication des fichiers fréquements demandé à travers un ensemble distribué de serveur

Gestion des Beans - Cache dans Spring

Activation de la mise en cache au niveau de l’application

```
@SpringBootApplication
@EnableCaching
public class SlideApplication {

    public static void main(String[] args) { SpringApplication.run(SlideApplication.class, args); }

}
```

Gestion des Beans - Cache dans Spring

Définition des éléments à mettre en cache dans les controllers

```
@RestController
public class CustomerController {

    @Cacheable(value = "customerInfo")
    public List<Customer> getAllAction() {
        return Arrays.asList(
                new Customer( id: 1L,  lastName: "Ovyn",  firstName: "Flavian", LocalDate.of( year: 1991,  month: 7,  dayOfMonth: 19)),
                new Customer( id: 2L,  lastName: "Trigalet",  firstName: "Maryse", LocalDate.of( year: 1992,  month: 12,  dayOfMonth: 17))
        );
    }
}
```

Gestion des Beans - Supervision

Gestion des Beans - supervision avec JMX

Java Management Extensions (JMX) fournit un mécanisme standard de monitoring et de gestion d’application.

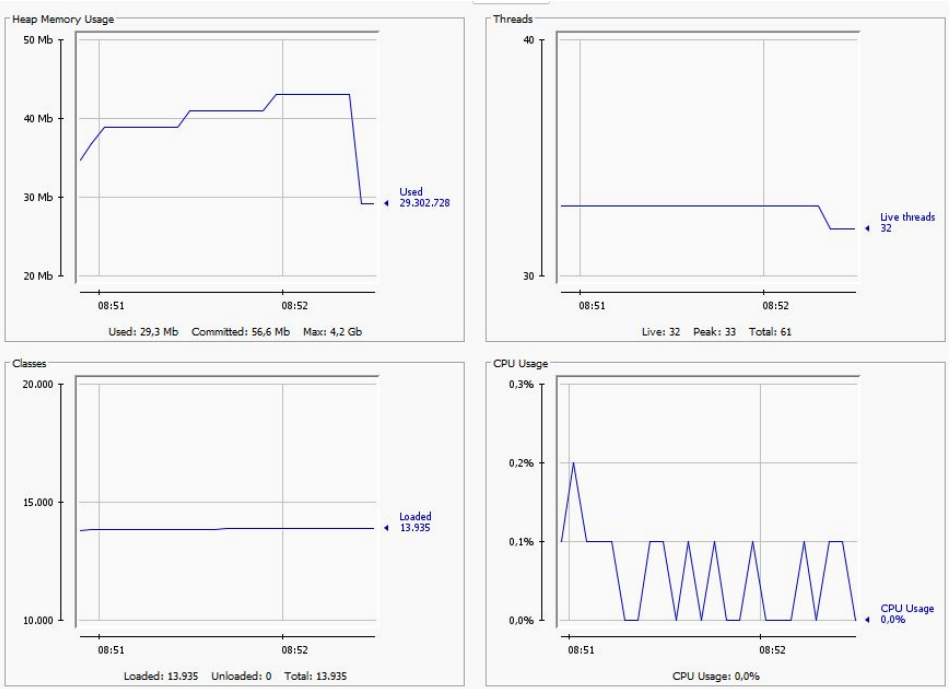
Dans l’application JConsole, tous les points d’accès se trouvent sous le domaine “org.springframework.boot”

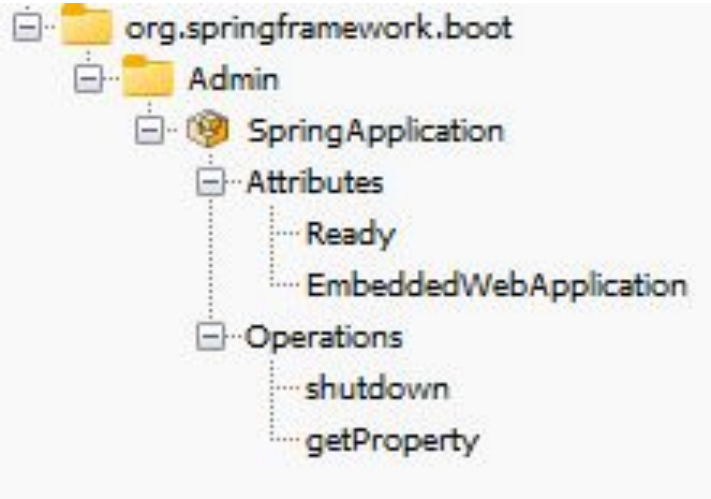
Par défaut, Spring possède un “Endpoint” JMX pure permettant d’obtenir des données de base (voir slide suivant). Il est conseillé d’utiliser le package Actuator

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-actuator</artifactId>
</dependency>
```

Gestion des Beans - JMX: JConsole sans Actuator

JMX sans actuator nous permet déjà un certain contrôle sur notre application



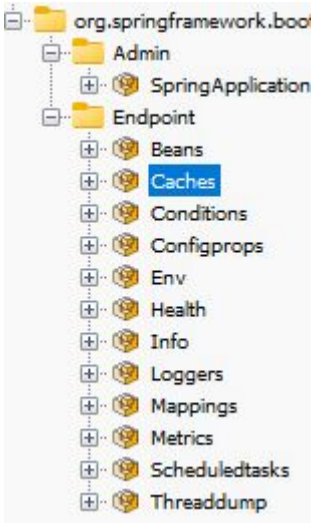


Gestion des Beans - JMX avec Actuator

Afin d’obtenir plus de points d'accès, il est possible de configurer le package Actuator.

Configuration:

```yaml
management:
  endpoints:
    web:
      exposure:
        include: "*"
    jmx:
      exposure:
        include: "*"
```



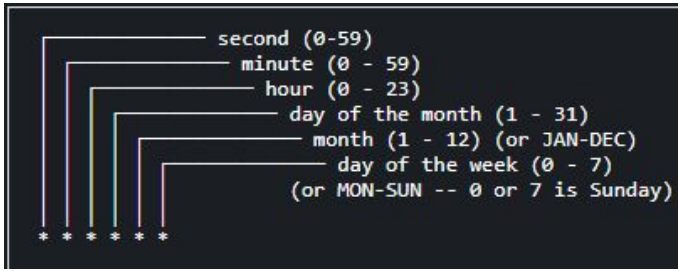
Gestion des Beans - Tâche planifiée

Gestion des Beans - Tâche planifiée

Spring permet l’exécution de tâches planifiées. Il s’agit de tâches qui doivent s'exécuter à intervalle régulier.

Il va pour cela utiliser des annotations ou de la configuration en XML et utiliser des CronExpression.

Une CronExpression est une chaîne de caractère qui dont chaque composant correspond à une notion de temps



Gestion des Beans - Tâche planifiée

Exemple de Cron Expression:

0 0 * * * *	Toutes les heures tous les jours
*/10 * * * * *	Chaque 10 secondes
0 0 8-10 * * *	8, 9, 10 heure tous les jours
0 0 6,19 * * *	6 et 19 heure tous les jours
0 0/30 8-10 * * *	Toutes les demi-heures entre 8h et 10h30 tous les jours
0 0 9-17 * * MON-FRI	De 9h à 17h tous les jours de la semaine
0 0 0 25 12 ?	Le jours de Noël à minuit

Gestion des Beans - Les profils

Gestion des beans - Les profils

Dans Spring, il est possible d’effectuer des configurations spécifiques liées au profil d’utilisation de l’application.

Ces différents profils sont accessible également dans les tests en utilisant l’annotation “@ActiveProfile”

Gestion des beans - Les profils

Afin de spécifier le profil à utiliser, il existe plusieurs manière de faire:

- Par programmation
- Par paramètre dans un fichier web.xml
- Par profil Maven
- Par le paramètre passer à la JVM
- Par variable d'environnement
  - application.properties
  - application.yml

Gestion des Beans - Le profile: Par config

Dans le fichier application.yml:

```yaml
spring:
  config:
    activate:
      on-profile: dev
---
spring:
  config:
    activate:
      on-profile: prod
```

Il est à noter l’importance des “---” qui permettent la séparation des différents environnements.

Il ne reste plus qu'à configurer le profile actif

Gestion des Beans - Le profile: Par config

Il existe également de base le fichier application.properties qui permet une configuration des variables d’environnement.

Afin de séparer différents environnements applicatifs, il est possible de créer des fichiers supplémentaires par environnement.

Ou de les séparer dans le fichier “application.properties” par un commentaire

```
spring.config.activate.on-profile=dev
#---
spring.config.activate.on-profile=prod
```

Gestion des Beans - Le profile: Par programmation

Dans un Bean de configuration de spring

```
@Configuration
public class EnvironmentInitializer implements WebApplicationInitializer {

    @Override
    public void onStartup(ServletContext servletContext) throws ServletException {
        servletContext.setInitParameter(
                s: "spring.profiles.active",  s1: "dev"
        );
    }
}
```

Gestion des Beans - Le profil: Par programmation

Dans les variables les variables d'environnement de l’application

```
@SpringBootApplication
public class SlideApplication {
    2 usages
    private final ConfigurableEnvironment environment;

    @Autowired
    public SlideApplication(ConfigurableEnvironment environment) {
        this.environment = environment;
        this.environment.setActiveProfiles("dev");
    }

    public static void main(String[] args) { SpringApplication.run(SlideApplication.class, args); }

}
```

Gestion des Beans - Le profile: Par XML

```xml
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
         version="3.1">

    <context-param>
        <param-name>spring.profiles.active</param-name>
        <param-value>dev</param-value>
    </context-param>
</web-app>
```

Gestion des Beans - Le profile: Par Maven

Déclaration des profils utilisateur dans le pom.xml

```
<profiles>
    <profile>
        <id>dev</id>
        <activation>
            <activeByDefault>true</activeByDefault>
        </activation>
        <properties>
            <spring.profiles.active>dev</spring.profiles.active>
        </properties>
    </profile>
    <profile>
        <id>prod</id>
        <properties>
            <spring.profiles.active>prod</spring.profiles.active>
        </properties>
    </profile>
</profiles>
```

Gestion des Beans - Le profil: Par Maven

Configuration du build permettant de remplacer le profil actif dans l'application

```
<build>
    <resources>
        <resource>
            <directory>src/main/resources</directory>
            <filtering>true</filtering>
        </resource>
    </resources>
</build>
```

Configuration dans le fichier application.properties

```
spring.profiles.active=@spring.profiles.active@
```

Gestion des Beans - Le profil

Test l’application avec un profil particulier.

Il est nécessaire d’annoter la class de test avec l’annotation “@ActiveProfiles”

```
@SpringBootTest
@ActiveProfiles("prod")
class SlideApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

Exercices

Gestion des Beans - Exercices

(Dans le cadre de l’exercice suivi, nous allons commencer par une application console afin de nous familiariser avec le framework)

Il vous est demandé dans cet exercice, de créer une application “n-tier”.

Votre application nécessite l’injection de dépendance pour réaliser les tâches métier.

Il sera également nécessaire d’inclure de l’ AOP afin d’automatiser différents cas métiers.

Accès aux données et gestion des transactions

Accès aux données - JDBC

Dépendance

Accès aux données - JDBC

```
@Repository
public class CustomerRepositoryImpl extends JdbcDaoSupport
```

```
@Autowired
public CustomerRepositoryImpl(NamedParameterJdbcTemplate jdbcTemplate) {
    this.jdbcTemplate = jdbcTemplate;
    setJdbcTemplate(this.jdbcTemplate.getJdbcTemplate());
}
```

Accès aux données - JDBC

```
1 usage
public static final String SELECT_ALL = "SELECT * FROM CUSTOMER";
1 usage
public static final String SELECT_BY_ID = "SELECT * FROM CUSTOMER WHERE ID = :id";
```

```
@Override
public List<Customer> findAll() { return jdbcTemplate.query(SELECT_ALL, this::mapRow); }

@Override
public Customer findOneById(Long id) {
    SqlParameterSource parameters = new MapSqlParameterSource()
            .addValue( paramName: "id", id);

    return jdbcTemplate.queryForObject(SELECT_BY_ID, parameters, this::mapRow);
}
```

Accès aux données - NoSQL

NoSQL pour Not Only SQL est un système de gestion de base de données non relationnel. Il répond à une partie des problématiques rencontrées dans le cadre du Big Data. Même si l’utilisation de cette technologie peut être utilisée hors du cadre du Big Data.

Les bases de données prisent en charge par Spring est:

- Redis
- MongoDB
- ElasticSearch
- Cassandra
- Geocode
- Couchbase
- Neo4j

Accès aux données - NoSQL: MongoDB

Activation de l’utilisation de repository de MongoDB au niveau de l’application

```
@EnableMongoRepositories
public class DataNosqlApplication
```

Accès aux données - NoSQL - MongoDB

Définition d’un document (Entity) et de son repository

```
3 usages
public interface BookRepository extends MongoRepository<Book, Integer> {

    1 usage
    @Query("{'bookName': ?0}")
    Book findByBookName(final String bookName);

    1 usage
    @Query("{'author.name':  ?0}")
    List<Book> findBooksByAuthorName(final String authorName);
}
```

```
12 usages
@Document
public class Book {
    @Id
    private String id;
    1 usage
    @NonNull private Integer bookId;
    1 usage
    @NonNull private String bookName;
    1 usage
    @NonNull private String bookAuthor;
    1 usage
    @NonNull private Double bookCost;
    1 usage
    private Author author;
```

Accès aux données - JPA

JPA pour Java Persistence Api (renommé Jakarta Persistence en 2019) est une spécification nécessitant un ORM (Object Relational Mapping).

Un ORM est un système implémentant les responsabilités de faire correspondre les objets Java au modèle relationnel.

- Stockage
- Récupération
- Mise à jours

Accès au données - JPA

Il existe plusieurs implémentations de l’API, les plus connus sont:

- Hibernate (Open Source)
- Top Link (Oracle)
- Eclipse Link (Eclipse)
- Open JPA (Open Source => iBATIS)

L’implémentation la plus populaire et la plus aboutie est Hibernate. Il a par conséquent la plus grande communauté et est bien documenté. C’est l’implémentation choisie par Spring

Accès aux données - JPA

Il est intéressant d'utiliser un ORM afin de réduire les problèmes liés aux différences de paradigme entre l’Orienté Objet et le modèle relationnel.

- Granularité
- Héritage
- Identité
- Associations
- Data Navigation

Accès au données - JPA

La granularité est la possibilité du système d'information à être répartie en plusieurs parties de taille plus petite.

Exemple:

Le détail d'une personne peut être rendu en créant deux classes

- une reprenant les informations de la personne
- une autre reprenant les informations de l'adresse.

On remarque dès lors une non correspondance de cette information avec le modèle relationnel

Accès au données - JPA

L’héritage et les sous-types est un concept naturel dans le cadre de l’OO. Cependant ce concept, bien que présent dans le schéma Entité-Association n’existe pas dans le modèle relationnel.

Il est donc nécessaire de faire des associations spécifiques permettant de rendre compte de ce concept.

- Création d’une seule table avec un discriminant
- Création de table pour les enfants uniquement
- Création d’une table pour le parent et d’une table pour chaque enfant

Accès aux données - JPA

L’identité d’un objet en OO est définie de plusieurs manières :

- l’égalité entre référence
- l’égalité en valeur.

Or il n’existe qu’une seule notion d’égalité dans le modèle relationnel: **la clé primaire**.

Accès aux données

Les associations sont également sources de différences.

En effet, en OO, une association est une référence unidirectionnel là où le modèle relationnel décrit une association comme étant bi-directionnel

Accès aux données - JPA

L’accès aux données en Java est fondamentalement différent que l’accès aux données au sein d’une base de données.

En Java, nous parcourons les associations en traversant chacun des objets alors qu’en SQL, on cherche à minimiser le nombre de requêtes en effectuant des jointures et en sélectionnant la cible avant de parcourir les informations du système

Accès aux données - JPA

Un ORM fournit un API permettant de résoudre, ou du moins réduire, les différentes problématiques citées précédemment.

L’ORM permet donc à l’application de manipuler les données comme des objets et non pas en utilisant les concepts comme Ligne, Colonnes, Tables.

Les caractéristique d’un ORM sont:

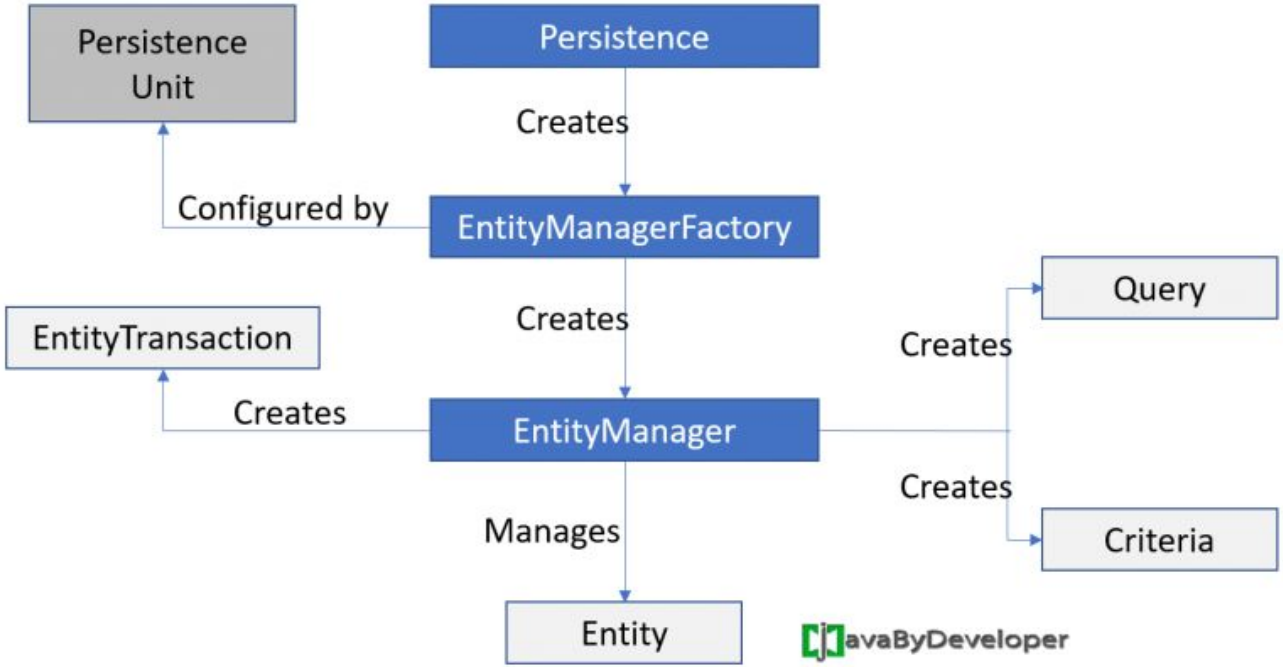
- Meilleur architecture
- Réduction de temps de développement
- Gestion de la cache et des transactions

Accès aux données - JPA

Fonctionnalité fournie par JPA:

- ORM Standardisé
- Héritage, Polymorphisme, Requête Polymorphique
- Annotation ou XML pour définir le mapping
- SQL-Like query language => JPQL et SQL natif
- Store Procedure
- Criteria API
- Bean Validation
- Caching

Accès aux données - JPA



Accès aux données - JPA: Entité

Une entité est un objet du domaine de persistance.

Chaque entité représente une table dans le modèle relationnel et chaque instance de l’entité représente une ligne de la table correspondante.

Accès aux données - JPA

L’EntityManager est une interface permettant l'interaction avec la base de données. Ces intéractions sont un CRUD

Chaque EntityManager est associé à un PersistenceContext. Un PersistenceContext gère l’ensemble des entités qui contient les données à persister

Accès aux données - JPA

```java
@Embeddable
public class Address {
    2 usages
    @Column(name = "address_street")
    private String street;
    2 usages
    @Column(name = "address_number")
    private String number;
    2 usages
    @Column(name= "address_city")
    private String city;
    2 usages
    @Column(name = "address_locality")
    private String locality;
```

```java
@Entity
public class Customer {
    1 usage
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY, generator = "native")
    @GenericGenerator(name = "native", strategy = "native")
    @Column(name = "customer_id")
    private Long id;
    4 usages
    @Column(name = "customer_lastname", nullable = false)
    private String lastName;
    4 usages
    @Column(name = "customer_firstname", nullable = false)
    private String firstName;
    1 usage
    @Column(name = "customer_birthdate")
    private LocalDate birthDate;

    @Embedded
    private Address address;
```

CUSTOMER_ID	ADDRESS_CITY	ADDRESS_LOCALITY	ADDRESS_NUMBER	ADDRESS_STREET	CUSTOMER_BIRTHDATE	CUSTOMER_FIRSTNAME	CUSTOMER_LASTNAME

Accès aux données - JPA: ManyToOne unidirectionnel

```java
@Entity
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private LocalDate orderDate;
    @ManyToOne
    @JoinColumn(name = "customer_id")
    private Customer customer;
}
```

@ManyToOne permet de définir une relation dont plusieurs commandes sont liés à un seul utilisateur

@JoinColumn permet de définir le nom de la colonne contenant la foreign key

Accès au données - JPA: ManyToOne Bidirectionnel

```
@OneToMany(mappedBy = "customer")
@MapKey(name = "orderDate")
private Map<LocalDate, Order> orders;
```

```
@Entity
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private LocalDate orderDate;
    @ManyToOne
    @JoinColumn(name = "customer_id")
    private Customer customer;
}
```

@OneToMany permet la déclaration d'une association OneToMany qui travaille de pair avec @ManyToOne. Le paramètre mappedBy permet de sélectionner le nom du champ dans l'entité propriétaire de la relation (*@ManyToOne*)

@MapKey (utiliser uniquement dans le cadre d'une map) permet la création d'une map dont la clé est de type LocalDate

Accès aux données - JPA: ManyToMany unidirectionnel

```
@ManyToMany
@JoinTable(name = "order_items", joinColumns = {
        @JoinColumn(name = "order_id", nullable = false),
}, inverseJoinColumns = { @JoinColumn(name = "product_id", nullable = false)})
private List<Product> products;
```

@*ManyToMany* permet la déclaration d'une association

@*JoinTable* permet la définition de la table de jointure

@*JoinColumn* permet la définition des métadonnées sur la colonnes de jointure

Accès aux données - JPA: ManyToMany bidirectionnel

```
@ManyToMany
@JoinTable(name = "order_items", joinColumns = {
        @JoinColumn(name = "order_id", nullable = false),
}, inverseJoinColumns = { @JoinColumn(name = "product_id", nullable = false)})
private List<Product> products;
```

```
@ManyToMany(mappedBy = "products")
private List<Order> orders;
```

Le paramètre *mappedBy* permet d'effectuer la liaison de la table jointe à la déclaration de la table

Accès aux données: JPA

```java
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String username;
    private String password;

    @OneToOne
    @JoinColumn(name = "customer_id")
    private Customer customer;
}
```

@OneToOne permet la définition d'une association unidirectionnel

Accès aux données - JPA: Lecture

```
List<Customer> findAll();
List<Customer> findAllByFirstName(String firstname);
List<Customer> findAllByFirstNameAndLastName(String firstname, String lastname);
@Query(value = "SELECT c FROM Customer c JOIN c.address a WHERE a.city = :city")
List<Customer> findAllCustom(@Param("city") String city);
2 usages
List<Customer> findAll(Specification<Customer> specification);
<T> Collection<T> findAll(Class<T> type);
```

Accès aux données: JPA - Procédure

```
@Query(value = "CALL storeProdecure :param1, :param2", nativeQuery = true)
List<Customer> callStoreProcedure(@Param("param1") String param1, @Param("param2") int age);
@Procedure(procedureName = "findAll")
List<Customer> callFindAllProcedure();
```

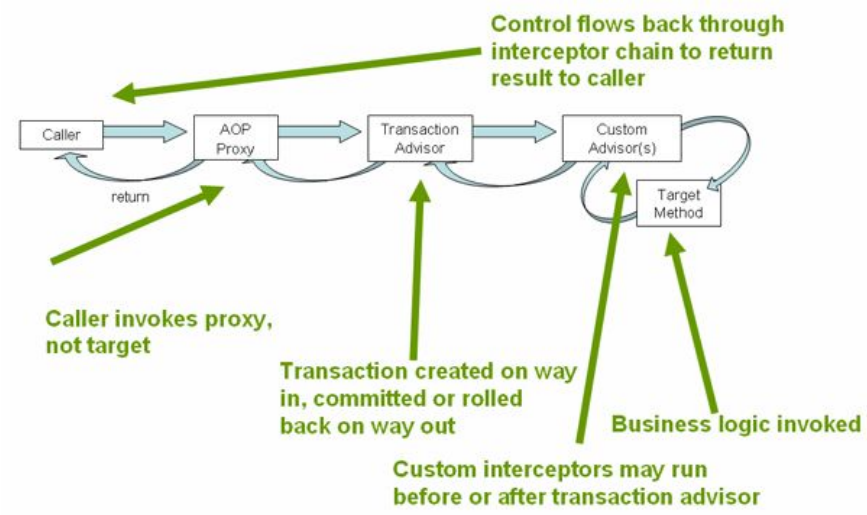
Accès aux données: JPA - Update

```
@Modifying
@Query(value = "UPDATE Customer c SET c.lastName = :lastname WHERE c.id = :id")
int setLastname(@Param("id") long id, @Param("lastname") String lastname);
```

Gestion des transactions

La gestion des transactions peut se faire via configuration XML ou annotation.

```xml
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource" destroy-method="close">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@rj-t42:1521:elvis"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
</bean>

<!-- similarly, don't forget the PlatformTransactionManager -->
<bean id="txManager" class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
    <property name="dataSource" ref="dataSource"/>
</bean>
```



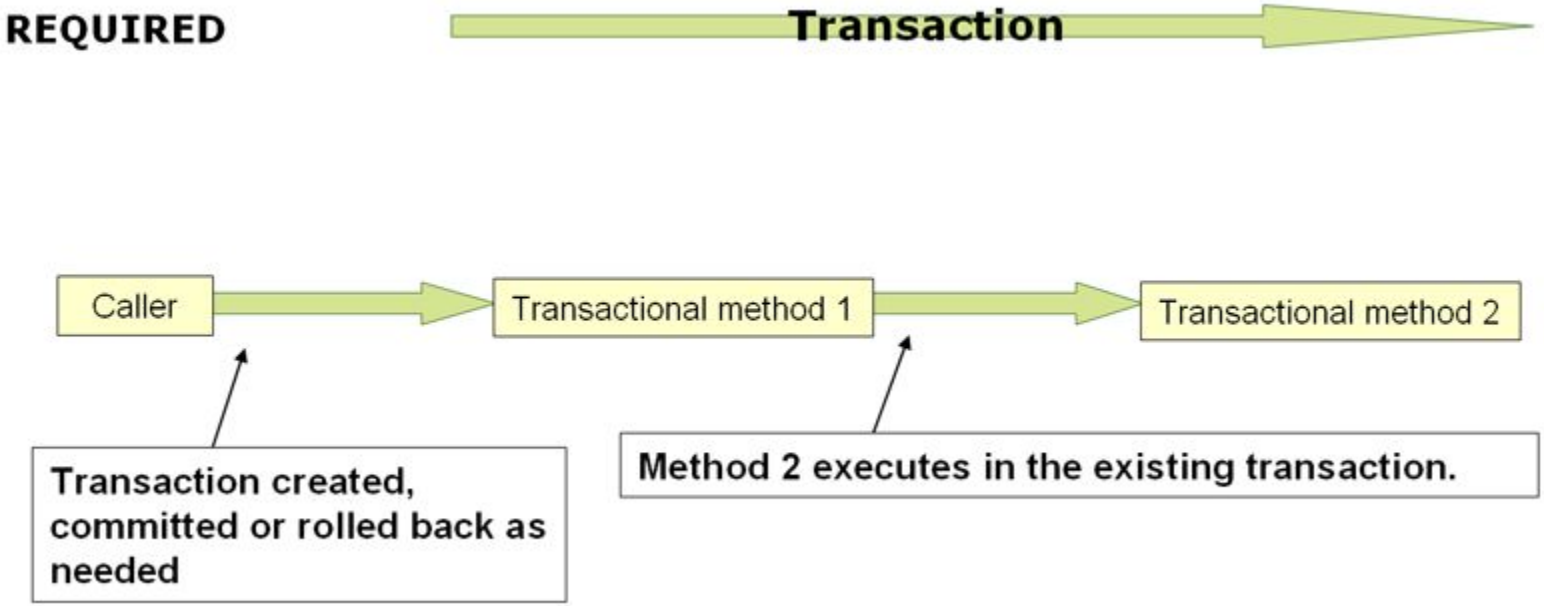
Gestion des transactions

L’annotation utiliser est @*Transaction*. Elle peut annoter aussi bien une classe qu’une méthode.

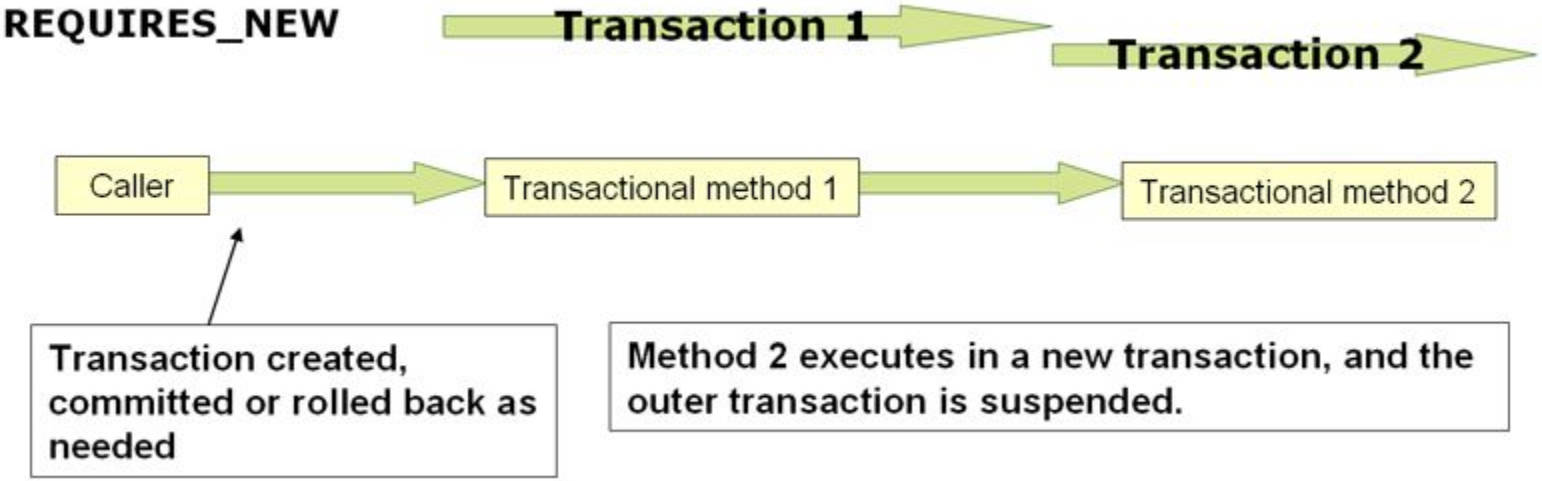
Elle a beaucoup de paramètre optionnel:

- Propagation
- Isolation
- ReadOnly
- Timeout
- RollbackFor && rollbackForClassName
- NoRollbackFor && noRollbackForClassName

Gestion des transactions: Propagation



Gestion des transactions: Propagation



Gestion des transactions - JDBC

```java
@Override
public List<Customer> insertMany(Collection<Customer> objs) {
    transactionTemplate.execute(new TransactionCallbackWithoutResult() {
        @Override
        protected void doInTransactionWithoutResult(TransactionStatus status) {
            try {
                objs.forEach(it -> insertOne(it));
                status.isCompleted();
            } catch (Exception e) {
                status.setRollbackOnly();
            }
        }
    });
    return objs.stream().toList();
}
```

Accès aux données et gestion des transactions

Exercice

Modifier l’application précédente afin d’obtenir une couche de données (DAL) permettant l’utilisation de la base de données avec JPA.

API REST avec Spring MVC et Spring Webflux

API REST avec Spring MVC - Bonne pratique

Conventions de nommage:

- Les URL doivent être en kebab-case
- Les paramètres et les champs de ressources doivent être en under_score ou camelCase
- Les noms de ressources doivent être en kebab-case pluriel

Exemple d'URL

```
/resources
/blogs

/resources/:resourceId
/blogs/:blogId

/resources/:resourceId/subresources
/blogs/:blogId/posts
```

API REST avec Spring MVC - Bonne pratique

Base URL

- Evitez les URLs complexe

Media Type

- application/json
- application/xml

API REST avec Spring MVC - Bonne pratique

Versioning

- Versioning par Media Type
- Versioning par URL
- Pas de versioning (solution à privilégier)

Propriété ID

- Uniformité

Polymorphisme

- Création d’un DTO avec des champs calculés.
- Création d’un champ discriminant

Remarque : L’abus de polymorphisme nuit à la santé de votre API

API REST avec Spring MVC - Bonne pratique

Datetime

- Format ISO 8601
  - YYYY-MM-DDThh:mm:ss.s(+hh:mm or -hh:mm)

Protocol HTTP

API REST avec Spring MVC - Protocole HTTP

HTTP pour Hypertext Transfer Protocol est un protocole de communication pour le WWW.
En 1989, Tim Berners-Lee (ingénieur et chercheur au CERN) a créé le protocole HTTP.

Le protocole contient des méthodes de connexions permettant d'interagir avec des API Rest basés justement sur ces méthodes

API REST avec Spring MVC - Protocole HTTP

Les différentes méthode du protocole et leur signification (code de retour):

- **GET** => Lecture (200, 404 Not Found)
- **POST** => Création (201, 404 Not Found, 409 Conflict si la ressource existe)
- **PUT** => Mise à jours complète, remplace (200, 204 No Content, 404)
- **PATCH** => Mise à jours partielle, modifié (200, 204, 404)
- **DELETE** => Suppression (200, 404)
- **OPTIONS** => Utiliser dans les pre-fligth request (Voir Securite - Cors)

Mise en place de l’API REST - Controller Advice

Un “Controller Advice” est une classe permettant l’interception de certaine exceptions afin de renvoyer au client un message personnalisé par rapport à l’erreur ainsi que de spécifier un code Http

```
@ControllerAdvice
public class PreConditionAdvice {

    @ExceptionHandler(value = PreConditionFailedException.class)
    public ResponseEntity<Object> preconditionHandler(PreConditionFailedException exception) {
        return new ResponseEntity<>(exception.getErrors(), HttpStatus.PRECONDITION_FAILED);
    }
}
```

Mise en place de l’API Rest

```
@RestController
@RequestMapping(path = {"/api/customers"})
public class CustomerController {
    4 usages
    private final CustomerRepository repository;

    @Autowired
    public CustomerController(CustomerRepository repository) {
        this.repository = repository;
    }
```

Mise en place d’une API REST

```
@GetMapping(path = {"", "/"})
public ResponseEntity<Page<Customer>> getAllAction(
        @RequestParam(name = "page", defaultValue = "1", required = false) int page,
        @RequestParam(name = "size", defaultValue = "5", required = false) int size
) {
    return new ResponseEntity<>(this.repository.findAll(PageRequest.of(page, size)), HttpStatus.OK);
}
@GetMapping(path = {"/{id}"})
public ResponseEntity<Customer> getOneAction(
        @PathVariable(name = "id") long id
) {
    Optional<Customer> oCustomer = this.repository.findById(id);
    if (oCustomer.isEmpty()) {
        throw new PreConditionFailedException("Customer with id "+ id+ " doesn't exist");
    }
    return new ResponseEntity<>(oCustomer.get(), HttpStatus.OK);
}
```

Mise en place d’une API REST

```java
@GetMapping(path = {"/address", "/{id}/address"})
public ResponseEntity<List<Address>> getAddresses(
        @PathVariable(name = "id", required = false) Long id,
        @RequestParam(name = "page", defaultValue = "1", required = false) int page,
        @RequestParam(name = "size", defaultValue = "5", required = false) int size
) {
    List<Address> addresses;
    if (id == null) {
        addresses = this.repository.findAll(PageRequest.of(page, size)).map(Customer::getAddress).stream().toList();
    } else {
        addresses = this.repository.findById(id).stream().map(Customer::getAddress).toList();
    }
    if (addresses.size() == 0) {
        throw new NoEntityException();
    }
    return new ResponseEntity<>(addresses, HttpStatus.OK);
}
```

Mise en place de l’API REST - Bean Validation

Grâce à la librairie de validation, Spring est capable de valider vos DTOs afin de n’exécuter le code que si il est nécessaire.

Il vous est également possible de récupérer les erreurs lors de la validation afin de renvoyer un message d’erreur adéquat.

```
@Valid @RequestBody CustomerCreate obj,
BindingResult bindingResult
```

Mise en place de l’API REST - Bean Validation

@NotNull

@NotBlank = @NotNull + @NotEmpty

@AssertTrue / @AssertFalse

@Size

@Length

@Max / @Min / @DecimalMin / @DecimalMax / @Digits

@Positive / @PositiveOrZero / @Negative / @NegativeOrZero

@Past / @PastOrPresent / @Future / @FuturOrPresent

@CreditCardNumber

@EAN

@Email

@Range

@SafeHtml

@ScriptAssert

@URL

@Pattern

Mise en place de l’API REST - Bean Validation

Ces annotations peuvent aussi être appliquées au type d’une collection, au type du Optional.

Cela nous permettra d’obtenir une validation des types de données de références plus pertinentes.

Il est fortement conseillé d’utiliser des DTOs pour effectuer ces validations afin d’alléger et de contrôler vos entités.

Mise en place de l’API REST - Bean Validation

Il est possible d’utiliser une controller advice afin de récupérer les erreurs et de les retourner proprement.

```java
@ControllerAdvice
public class ValidationAdvice {
    @ExceptionHandler(value = {MethodArgumentNotValidException.class})
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    @ResponseBody
    public ValidationErrorDTO methodArgumentNotValidHandler(MethodArgumentNotValidException e) {
        BindingResult br = e.getBindingResult();
        List<FieldError> fieldErrors = br.getFieldErrors();
        ValidationErrorDTO dto = new ValidationErrorDTO();

        fieldErrors.forEach(it -> dto.addFieldError(it.getField(), it.getDefaultMessage()));
        return dto;
    }
}
```

Mise en place de l’API REST

```java
@PostMapping(path = {"", "/"})
public ResponseEntity<Customer> addOneAction(
        @Valid @RequestBody CustomerCreate obj,
        BindingResult bindingResult
        ) {
    if (bindingResult.hasErrors()) {
        Collection<String> errors = bindingResult.getAllErrors().stream().map(ObjectError::getDefaultMessage).toList();
        throw new PreConditionFailedException(errors);
    }

    return new ResponseEntity<>(this.repository.save(obj.toEntity()), HttpStatus.CREATED);
}
```

Mise en place d’une API REST

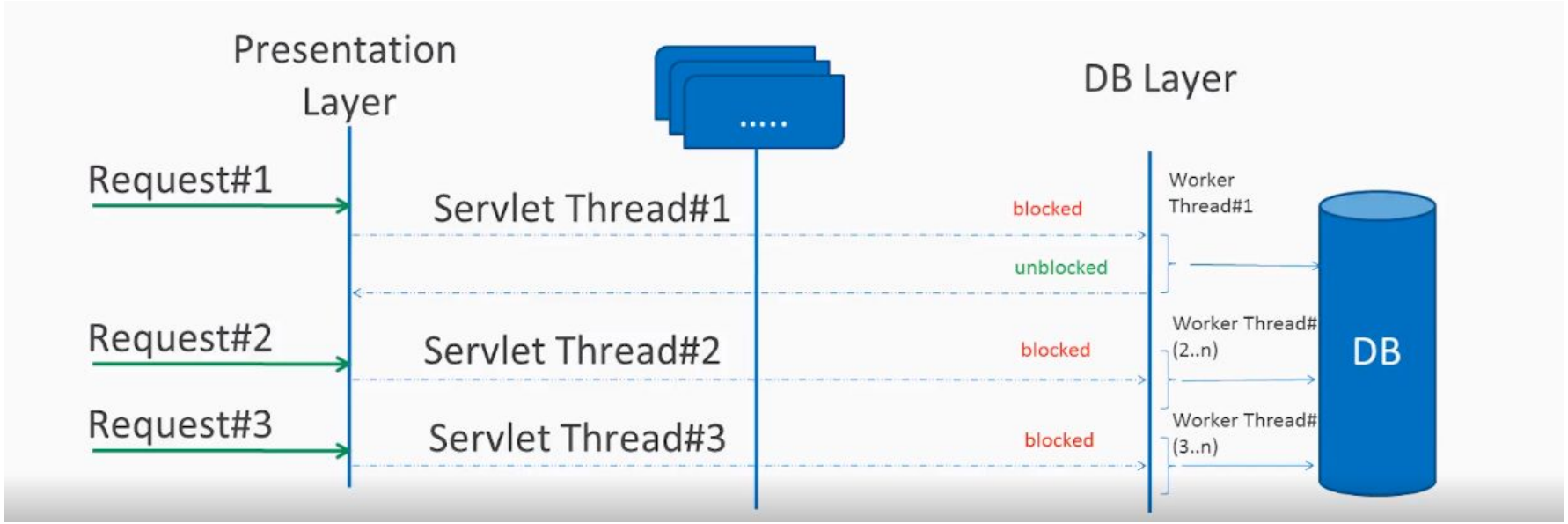
```java
@PutMapping(path = {"/{id}"})
public ResponseEntity<Customer> updateAddressAction(
        @PathVariable("id") long id,
        @RequestBody Customer newCustomer
) {
    newCustomer.setId(id);
    return new ResponseEntity<>(this.repository.save(newCustomer), HttpStatus.ACCEPTED);
}

@PatchMapping(path = {"/{id}/address"})
public ResponseEntity<Customer> updateAddressAction(
        @PathVariable("id") Customer customer,
        @RequestBody Address address
) {
    customer.setAddress(address);
    this.repository.save(customer);
    return new ResponseEntity<>(customer, HttpStatus.OK);
}
```

Mise en place d’une API REST

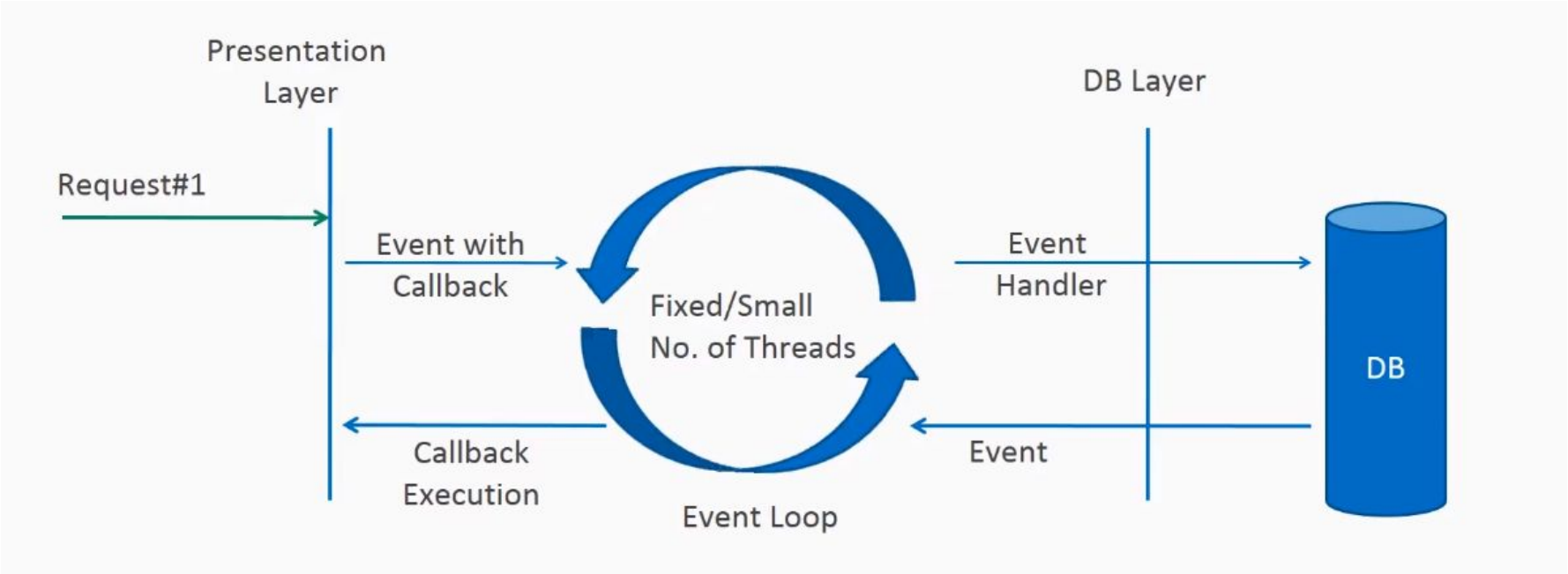
```
@DeleteMapping(path = {"/{id}"})
public ResponseEntity<Void> deleteAction(
        @PathVariable("id") long id
) {
    this.repository.deleteById(id);

    return ResponseEntity.accepted().build();
}
```

WebFlux

Mise en place de WebFlux



Mise en place de WebFlux



Mise en place de WebFlux

Mise en place d’une API REST et WebFlux

Exercice

Créer une application Web exposant les services développés précédemment.
Création d’une consommation de l’API avec un client HTML/Javascript

Sécurité

Sécurité: Dépendance dans Spring Boot

```
<dependency>
    <groupId>org.springframework.boot</groupId>    You, 20/
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Sécurité: Dépendance sans Spring Boot

Minimum:

```
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-web</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-config</artifactId>
</dependency>
```

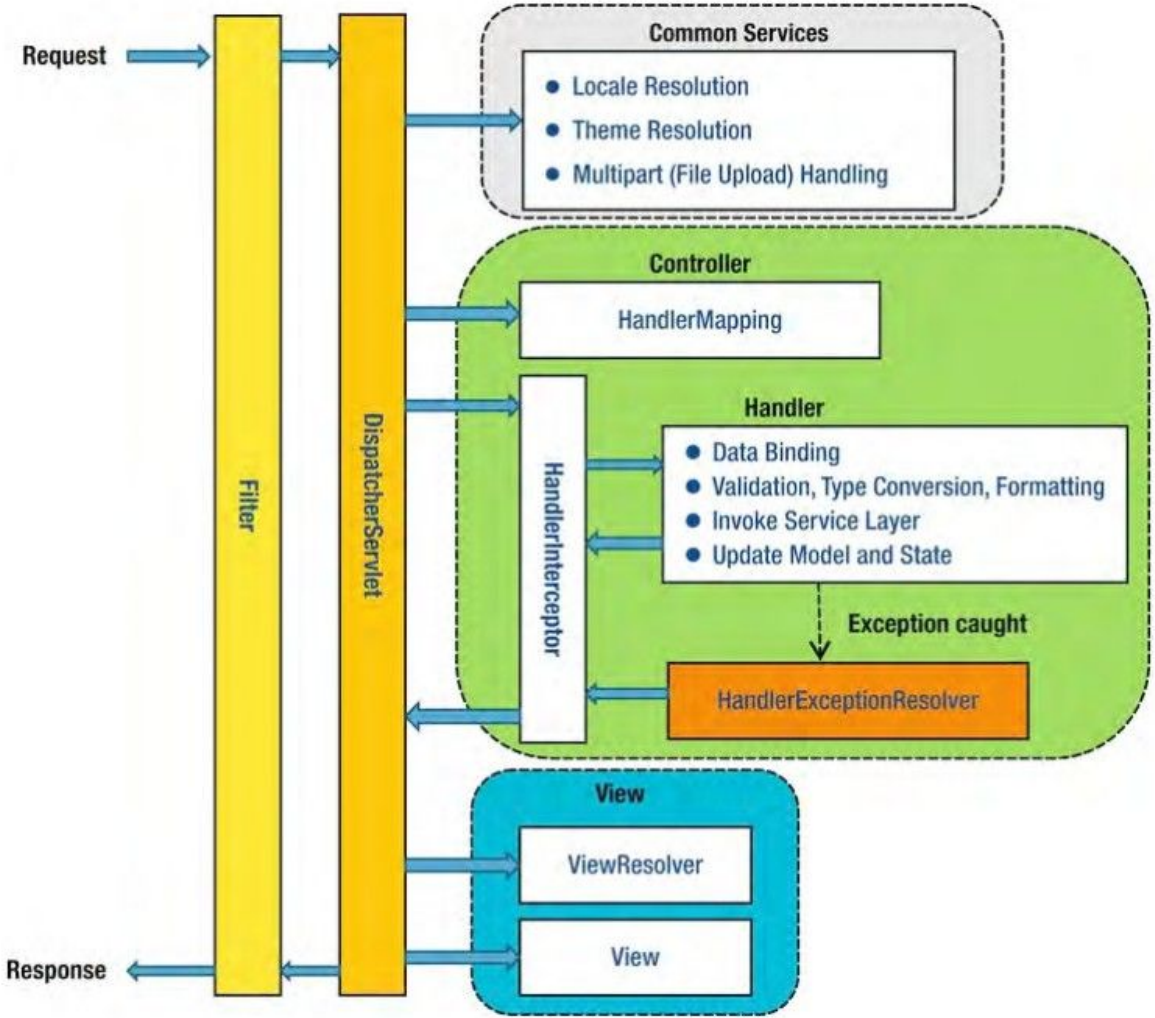


Figure 17-3. Spring MVC request life cycle